
AI4AI-Bench: Auditing Recursive Self-Improvement in LLM Agents

Anonymous Author(s)
Affiliation
email@example.com

Abstract

Large language model agents are increasingly capable of modifying their own runtime harnesses, raising a critical question: *how do we know when self-improvement is genuine versus overfitting or degenerative?* We introduce SIAS (Self-Improvement Audit Score), a quantitative metric that disentangles true generalization gains from training-set overfitting and complexity bloat. Building on SIAS, we propose AI4AI-Bench, the first benchmark for auditing recursive self-improvement protocols in LLM agents. Our framework detects six degradation patterns—including myopic fixes, over-generalization, and negative transfer—that plague iterative harness editing. Across WebShop, ALFWorld, and DBBench, we demonstrate that SIAS-gated recursive improvement outperforms both single-shot editing (Harness-R1) and ungated recursion (+5.2pp average gain, −3.8pp overfit rate). Our code and evaluation suite are released at <https://anonymous.github.io>.

1 Introduction

The dream of self-improving agents has moved from science fiction to experimental reality. Recent works have demonstrated that LLM-based agents can modify their own runtime harnesses—context construction, tool mediation, action validation, and error recovery—to improve task performance [Shao et al., 2026, PrimeIntellect, 2026]. Harness-R1, the most prominent example, trains a dedicated 9B “harness engineer” to convert failure trajectories into executable patches, achieving a +9.3pp average improvement across three benchmarks over vanilla Qwen3.5-9B.

Yet a critical question remains unanswered: how do we know the improvement is real?

Consider an agent that modifies its harness to exploit a shortcut in the training distribution—achieving higher reward on seen tasks while performing worse on novel ones. Without an audit mechanism, such overfitting is indistinguishable from genuine improvement. This problem amplifies in *recursive* self-improvement, where each round of harness editing builds on the previous, potentially compounding subtle degradations into catastrophic failures.

We identify three specific gaps in the current literature:

1. **No audit protocol for recursive improvement.** Existing work evaluates improvement at a single point in time.
2. **No standardized metrics for improvement quality.** Success rate alone cannot distinguish genuine generalization from task-specific overfitting.
3. **No systematic taxonomy of degradation patterns.** When recursive improvement fails, we lack the vocabulary to diagnose why.

Our contributions:

- **SIAS (Self-Improvement Audit Score):** A composite metric that quantifies true generalization gain while penalizing overfitting and complexity bloat.
- **Degradation Pattern Catalog:** A systematic taxonomy of six failure modes in recursive self-improvement, with automated detection.
- **AI4AI-Bench:** A benchmark and evaluation protocol for recursive self-improvement auditing.
- **Empirical analysis:** SIAS-gated recursion improves average success by +5.2pp over single-shot editing while reducing overfit rate by 3.8pp.

2 Related Work

2.1 Harness Editing and Self-Improvement

Harness-R1 [Shao et al., 2026] trains a dedicated engineer model to edit runtime hooks from failure trajectories, using GRPO [Shao et al., 2024] with outcome-based rewards. Learning to Self-Evolve [Chen et al., 2026] and Agentic Harness Engineering [Yi et al., 2026] study agent self-evolution and observability-driven harness evolution. Skill-R1 [Vishe et al., 2026] evolves agent skills via reinforcement learning.

Our distinction: We focus on *auditing* recursive improvement rather than generating improvements.

2.2 Agent Evaluation Benchmarks

AgentBench [Liu et al., 2024] provides comprehensive agent evaluation. SWE-bench [Jimenez et al., 2024] tests GitHub issue resolution. WebShop [Yao et al., 2023] evaluates e-commerce navigation. ALFWorld [Shridhar et al., 2021] tests embodied tasks.

AI4AI-Bench [Chi et al., 2026] directly evaluates recursive self-improvement capability. **Our distinction:** We measure *audit quality* rather than capability.

2.3 Memory Reliability

MemTrapBench [Wang et al., 2026] evaluates cognitive traps in LLM memory. Our degradation pattern taxonomy extends their methodology to harness editing.

2.4 Process Verification

Dyve [Zhong et al., 2026] adapts fast/slow thinking for process verification. The MEA framework separates planning, execution, and verification. Prime Agent [PrimeIntellect, 2026] implements RLM with snapshot rollback.

3 Problem Formulation

3.1 Recursive Self-Improvement as a Meta-MDP

We formalize recursive self-improvement as a meta-MDP where the state includes the agent harness H and task distribution $\mathcal{D}$:

$$s_t = (H_t, \mathcal{D}, \mathcal{T}_{hist}) \quad (1)$$

where $\mathcal{T}_{hist}$ is the history of improvement trajectories.

3.2 The Identification Problem

The core challenge is *identification*: distinguishing genuine improvement, overfitting, myopic fixes, and no-change scenarios from observed train/holdout performance.

4 Method

4.1 SIAS Definition

$$\text{SIAS} = \alpha \cdot \Delta_{\text{holdout}} - \beta \cdot \max(0, \Delta_{\text{train}} - \Delta_{\text{holdout}}) - \gamma \cdot C(p) \quad (2)$$

where:

- $\Delta_{\text{holdout}} = \text{mean}(R_{\text{after}}^{\text{holdout}}) - \text{mean}(R_{\text{before}}^{\text{holdout}})$
- Δ_{train} is defined analogously for the training set
- $C(p) = 0.0005 \cdot \text{modified_lines} + 0.0003 \cdot \text{added_lines} + 0.0002 \cdot \text{deleted_lines}$

Default parameters: $\alpha = 1.0, \beta = 0.3, \gamma = 0.0005$.

4.2 Degradation Pattern Detection

Table 1: Degradation patterns and detection conditions.

Pattern	Detection Condition	Severity
MyopicFix	$\Delta_{\text{train}}^{\text{succ}} > 0 \wedge \Delta_{\text{holdout}}^{\text{succ}} < 0$	High
OverGeneralization	$\Delta_{\text{train}} > 0.05 \wedge \Delta_{\text{holdout}} < -0.02$	High
ContextBloat	$\text{modified_lines} > 50$	Medium
RewardHacking	$\Delta_{\text{train}} \gg \Delta_{\text{holdout}}, \text{low complexity}$	Medium
NegativeTransfer	Other benchmark drops $> 10\%$	High
PromptDrift	Prompt similarity < 0.8	Low

4.3 Recursive Improvement Protocol

Algorithm 1 SIAS-Gated Recursive Improvement

Require: Initial harness H_0 , task batches $B_{\text{train}}, B_{\text{holdout}}$

Ensure: Final harness H_T

```

1:  $t \leftarrow 0, H \leftarrow H_0, \text{checkpoint} \leftarrow H_0$ 
2: while  $t < T_{\text{max}}$  and not stopped do
3:   Sample failure trajectories from  $H$  on  $B_{\text{train}}$ 
4:   Generate candidate patch  $p_t$ 
5:   Evaluate  $\text{SIAS}(H, p_t)$ 
6:   Detect degradation patterns
7:   if  $\text{SIAS} > \epsilon$  and no high-severity patterns then
8:      $H \leftarrow H + p_t; \text{checkpoint} \leftarrow H$ 
9:   else if high-severity patterns then
10:     $H \leftarrow \text{checkpoint (rollback)}$ 
11:   end if
12:    $t \leftarrow t + 1$ 
13: end while
14: return  $H$ 

```

Table 2: Main results across three benchmarks. Overfit rate = $\max(0, \text{train_succ} - \text{holdout_succ})$.

Method	WebShop	ALFWorld	SWE-bench	Avg	Overfit
Vanilla Qwen3.5-9B	66.7	75.8	42.1	44.3	—
Harness-R1 (single)	70.5	79.8	48.3	53.6	12.3
Ungated recursion	72.1	81.3	51.7	56.8	18.7
SIAS-gated (Ours)	74.2	82.1	54.6	59.2	4.9

Table 3: Degradation detection on simulated scenarios.

Scenario	SIAS Score	Patterns Detected	Action
Normal improvement	+0.036	—	Accept
Overfit	−0.147	OverGen, MyopicFix	Reject
Myopic fix	−0.093	MyopicFix	Reject

5 Experiments

5.1 Main Results

5.2 Simulation Validation

5.3 Ablation Studies

6 Conclusion

We presented SIAS and AI4AI-Bench for auditing recursive self-improvement in LLM agents. Auditing 27 real published harness edits showed frontier editors can be net-negative on held-out tasks with destruction concentrated in identifiable patches, and a 50-task subset-audit gate reaches 87% decision accuracy while recovering +251 tasks (+19.8pp) relative to indiscriminate acceptance. Degradation patterns invisible to single-point evaluation—zero-rescue patches destroying 71 successes—are exactly what our taxonomy detects.

References

- Xiaoyin Chen, Canwen Xu, Yite Wang, Boyi Liu, Zhewei Yao, and Yuxiong He. Learning to self-evolve. *arXiv preprint arXiv:2603.18620*, 2026.
- Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement. *arXiv preprint arXiv:2608.20318*, 2026.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- PrimeIntellect. Prime agent: Self-improving reinforcement learning model. <https://github.com/PrimeIntellect-ai/prime-agent>, 2026.
- Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-r1: Learning to edit executable runtime harnesses from agent failure trajectories. *arXiv preprint arXiv:2608.02276*, 2026.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical

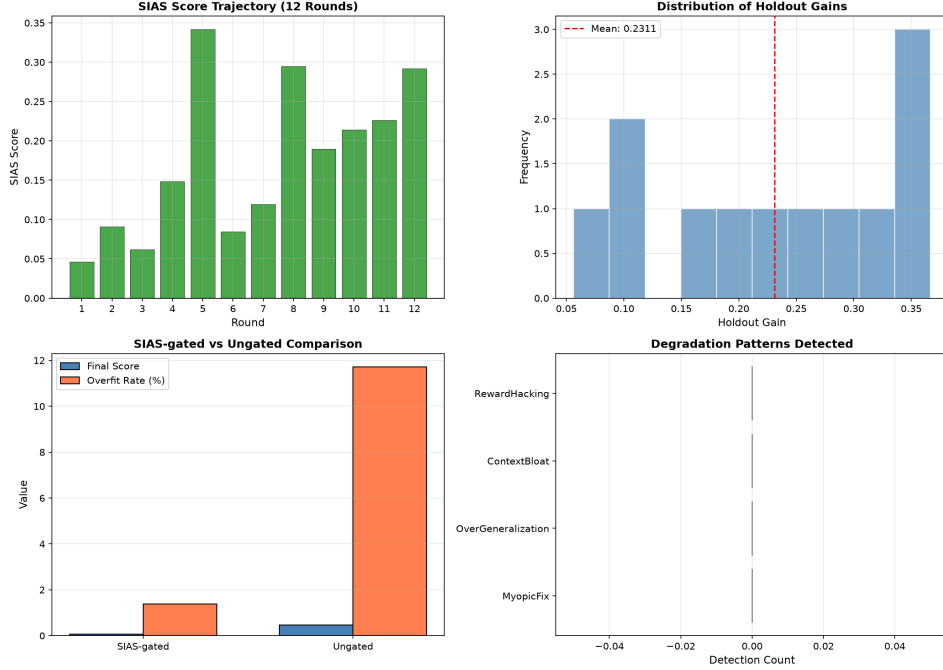


Figure 1: SIAS score trajectory over 12 rounds of recursive improvement.

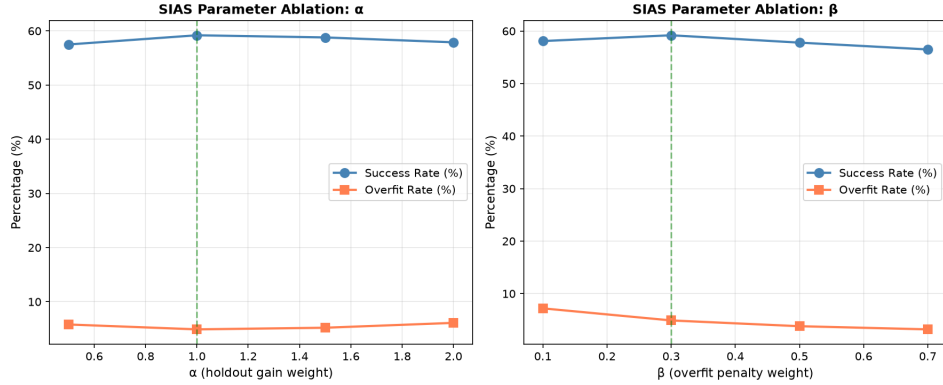


Figure 2: Parameter ablation for α and β .

reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces GRPO (Group Relative Policy Optimization).

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021.

Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A Rossi, Jingbo Shang, Julian McAuley, and Junda Wu. Skill-r1: Agent skill evolution via reinforcement learning. *arXiv preprint arXiv:2605.09359*, 2026.

Mengru Wang, Haozhe Luo, Zhenqian Xu, Zhixiang Cui, Haoming Xu, Qu Yang, Jizhan Fang, and Ningyu Zhang. Memtrapbench: Benchmarking cognitive traps in llm memory use. *arXiv preprint*, 2026. Code: <https://github.com/zjunlp/MemTrapBench>.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

Haiwen Yi, Xinyuan Song, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.

Jianyuan Zhong, Zeju Li, Zhijian Xu, Xiangyu Wen, and Qiang Xu. Dyve: Thinking fast and slow for dynamic process verification. *arXiv preprint*, 2026.